



Es. 1

- Un interprete è un programma I^{L_0} che prende un programma P^L (scritto in L) e lo esegue. È un programma generico scritto in L_0 , prende anche l'input quando serve.

- $\forall P^L \in \text{Prog.} \quad I^{L_0}(P^L \times \text{input}) \rightarrow \text{output}$
 $P^L(\text{input}) \rightarrow \text{output}$
input, output $\in D$
↓
insieme di dati manipolati

- begin

go := true;

while go do

FETCH(opcode, opinfo) at PC;

DECODE(opcode, opinfo);

if opcode needs arguments then

FETCH(args);

case opcode of

op1 : EXECUTE(op1, args);

op2 : EXECUTE(op2, args);

...

HALT : go := false;

if opcode has result then

STORE(result);

PC += size(opcode);

// legge riga di codice
// dove indica Program Counter
// decodifica il codice

// esegue il codice e
// nel caso fosse un
// HALT modifica go
// in false per
// concludere l'esecuzione

// passa alla riga di
// codice successiva

Es. 2

$$\bullet \in S \quad s_1, s_2 \in S \quad (s_1 \oplus s_2) \in S \quad (s_1 \otimes s_2) \in S$$

$$\begin{cases} n(\bullet) = 1 \\ n(s_1 \oplus s_2) = n(s_1 \otimes s_2) = n(s_1) + n(s_2) \end{cases}$$

$$\begin{cases} o(\bullet) = 0 \\ o(s_1 \oplus s_2) = o(s_1 \otimes s_2) = o(s_1) + o(s_2) + 1 \end{cases}$$

$$\forall s \in S \quad n(s) = o(s) + 1$$

Caso base

$$s = \bullet \quad n(s) = o(s) + 1 \stackrel{\text{def}}{\Rightarrow} 1 = 0 + 1 \Rightarrow 1 = 1 \quad \checkmark$$

Passo induttivo

$$\text{Hp. ind.} : n(s) = o(s) + 1$$

~~assunto~~

$$\text{per } s = (s_1 \oplus s_2) \quad \text{analogo per } s = (s_1 \otimes s_2)$$

$$n(s_1 \oplus s_2) = o(s_1 \oplus s_2) + 1 \quad ?$$

$$\begin{aligned} n(s_1 \oplus s_2) &\stackrel{\text{def}}{=} n(s_1) + n(s_2) \stackrel{\text{Hp. ind.}}{=} o(s_1) + 1 + o(s_2) + 1 = \\ &= o(s_1 \oplus s_2) + 1 \quad \checkmark \\ &\stackrel{\text{def}}{=} \end{aligned}$$

Es. 3

Lo scoping si usa quando in un programma bisogna risolvere i riferimenti a variabili non locali o globali e si dividono in:

- Scoping statico = ambiente valido al momento della definizione
tempo di ~~comp~~ compilazione
- Scoping dinamico = ambiente valido al momento della chiamata
tempo di interpretazione

- link statico è la catena che collega due o più funzioni al momento della definizione.

~~questo momento~~ $K = Sd(\text{chiamante}) + Sd(\text{chiamato}) + 1$

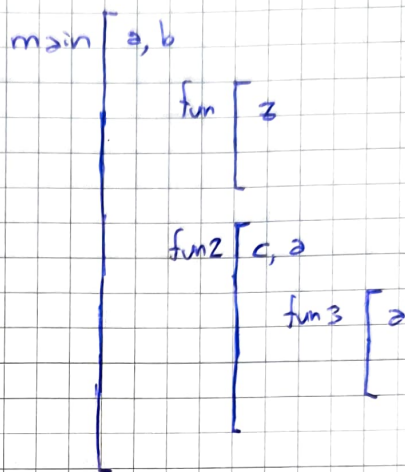
$Sd = \begin{matrix} \text{profondità} \\ \text{dichiarazione} \end{matrix}$

per risolvere le variabili il calcolo è (trovare RDA) :

$$N = Sd(\text{chi usa la variabile}) - Sd(\text{ultimo dichiarante})$$

- link dinamico è la catena che segue l'ordine delle chiamate.

```
{ int a=2; int b=3;
  void fun() {
    int z=a*b; }
  void fun2(int a) {
    int c=5;
    void fun3() {
      int a=c-b;
      fun(); }
    b = 2a+b;
    fun3(); }
  fun2(4); }
```



Profondità dichiarazioni

$$Sd(\text{main}) = 0$$

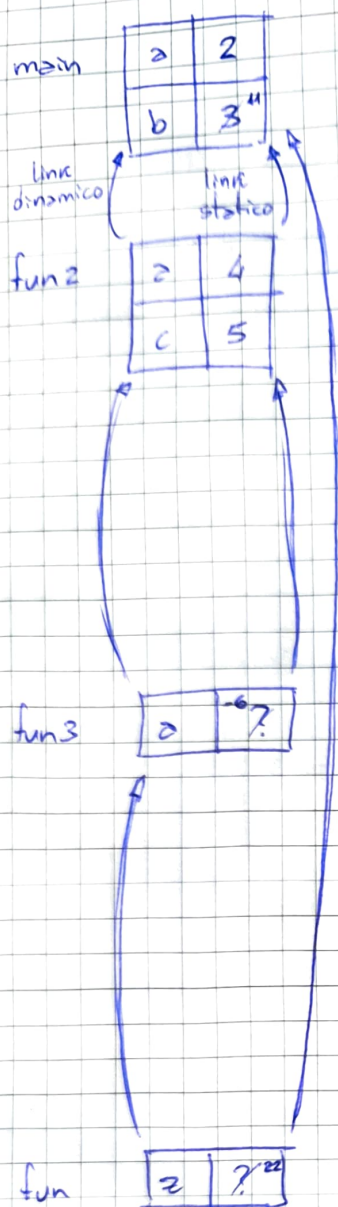
$$Sd(\text{fun}) = Sd(\text{fun2}) = 1$$

$$Sd(\text{fun3}) = 2$$

Sequenza chiamate

main $\rightarrow$ fun2 $\rightarrow$ fun3 $\rightarrow$ fun

Scoping statico



$$\text{link statico fun2} = K_{\text{fun2}} = \text{Sd}(\text{main}) - \text{Sd}(\text{fun2}) + 1$$

$$CS(\text{fun2}) = \text{main} \leftarrow = 0 - 1 + 1 = 0$$

esecuzione di fun2 : $b = 2a + b;$

- a è locale = 4
- $b \Rightarrow N_b = \text{Sd}(\text{fun2}) - \text{Sd}(\text{main}) = 1 - 0 = 1$

prendiamo b dal main : $b = 2 \cdot 4 + 3$

$$K_{\text{fun3}} = \text{Sd}(\text{fun2}) - \text{Sd}(\text{fun3}) + 1 = 1 - 2 + 1 = 0$$

$$CS(\text{fun3}) = \text{fun2}$$

exe di fun3 : $a = c - b;$

- a locale
- $c \Rightarrow N_c = \text{Sd}(\text{fun3}) - \text{Sd}(\text{fun2}) = 2 - 1 = 1$ prendiamo c da fun2
- $b \Rightarrow N_b = \text{Sd}(\text{fun3}) - \text{Sd}(\text{main}) = 2 - 0 = 2$ prendiamo b da main

$$K_{\text{fun}} = \text{Sd}(\text{fun3}) - \text{Sd}(\text{fun}) + 1 = 2 - 1 + 1 = 2$$

$$CS(\text{fun}) = \text{main}$$

exe di fun : $z = a * b;$

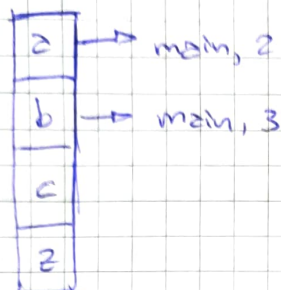
- z locale
- $a \Rightarrow N_a = \text{Sd}(\text{fun}) - \text{Sd}(\text{main}) = 1 - 0 = 1$
- $b \Rightarrow N_b = \text{Sd}(\text{fun}) - \text{Sd}(\text{main}) = 1 - 0 = 1$ prendiamo a, b da main

Finita l'esecuzione non ci sono più chiamate e quindi il programma termina deallocando tutta la memoria.

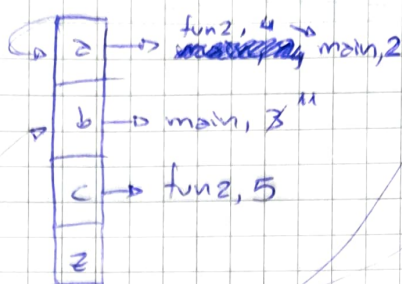


Scoping dinamico

main



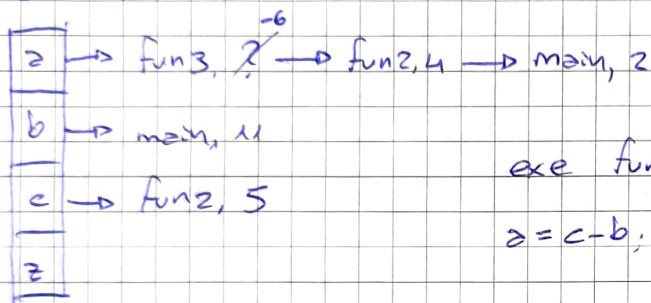
main → fun 2



exe di fun 2

$$b = 2a + b$$

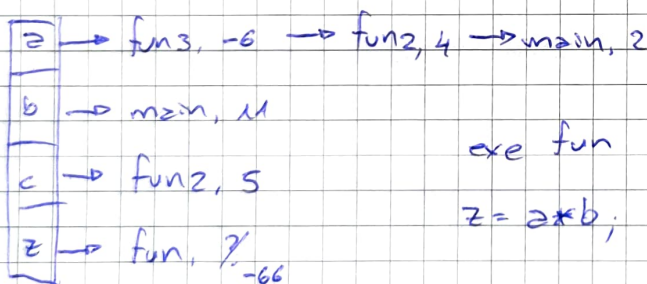
fun 2 → fun 3



exe fun 3

$$a = c - b; \Rightarrow a = 5 - 11 = -6$$

fun 3 → fun



exe fun

$$z = a * b; \Rightarrow z = -6 * 11 = -66$$

A differenza dello scoping statico $z = -66$ invece che $z = 22$

Es 4

- Per avere ~~una~~ valori differenti tra gli scoping, la funzione `fun()` deve essere dichiarata in `(*)`, ovvero all'esterno di `fun()`.
- La variabile `x` deve essere dichiarata dentro `fun()` e che cambi durante il `while`.

`(*) = int fun() { return x; }`

~~`(**) = int x = i;`~~

```
{ int i=0; int x:=3;  
  int fun() { return x; }  
  mod fun() {
```

```
    int y;
```

```
    int x=i;
```

```
    x:=fun();
```

```
    i:=i+2; }
```

```
while (i<2) { fun(); } }
```

- Durante lo scoping statico, `fun()` ritorna sempre 3 perché prende quello del `main`.

- Durante lo scoping dinamico, `fun()` ritorna `i` perché prende l'ultima `x` dichiarata ovvero in `fun()`.
`i` cambia ad ogni ciclo quindi sarà sempre diversa.

Es 5

- Una funzione è ricorsiva se è definita in funzione di sé stessa. La ricorsione è un metodo alternativo all'iterazione. Una funzione è ricorsiva in coda se restituisce ~~un~~ il risultato senza ulteriori computazioni di una funzione ricorsiva. Dettaglio importante è la gestione di memoria in quanto quella in coda non ha operazioni in attesa in "passato" e quindi l'`RdA` è sovrascritto e non appunto.


```
lista function (lista list) {
```

```
  if (length(list)=0) then return ();
```

```
  else return concat (function (cdr(list)), car(list)+2); }
```

- esecuzione con list = (2, 5, 8)

function ((2, 5, 8)) → concat (function ((5, 8)), 2+2)

(10, 7, 4)

concat (function ((8)), 5+2)

(10, 7)

concat (function ((1)), 8+2)

(10)

if → ()

⇒ la funzione inverte la lista e somma 2 a tutti gli elementi

- lista fun-rc (lista list) { return fun (list, ()); }

~~return fun (list, ()); }~~

↳ caso in cui list fosse vuota viene restituito ()

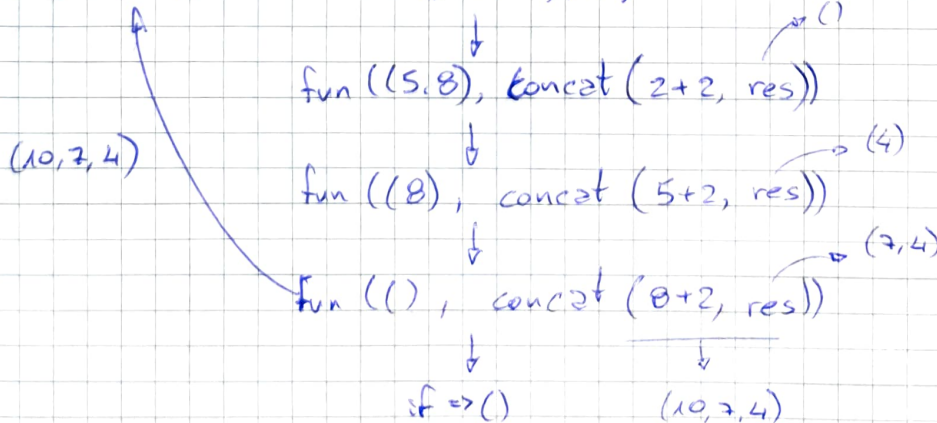
```
lista fun (lista list, lista res) {
```

```
  if (length(list)=0) return res;
```

```
  else return fun (cdr(list), concat(car(list)+2, res)); }
```

- exe con list = (2, 5, 8)

fun-rc ((2, 5, 8)) → fun ((2, 5, 8), ())



Es. 6

$p = [x \mapsto 5, y \mapsto ly]$

x costante, y variabile

$\sigma = [ly \mapsto 3]$

caso true

$p \vdash \langle y < 4, \sigma \rangle \xrightarrow{*} \langle \text{true}, \sigma \rangle$

$p \vdash \langle \text{while } y < 4 \text{ do } \{y := x\}, \sigma \rangle \xrightarrow{*} \langle y := x, \sigma \rangle$

$p(y) = ly$

$\sigma(ly) = 3$

caso false

$p \vdash \langle y < 4, \sigma \rangle \xrightarrow{*} \langle \text{false}, \sigma \rangle$

$p \vdash \langle \text{while } y < 4 \text{ do } \{y := x\}, \sigma \rangle \xrightarrow{*} \langle \cdot, \sigma \rangle$

$p(y) = ly$

$\sigma(ly) = 3$

analisi condizione

$p \vdash \langle y, \sigma \rangle \xrightarrow{*} \langle 3, \sigma \rangle$

$p \vdash \langle y < 4, \sigma \rangle \xrightarrow{*} \langle 3 < 4, \sigma \rangle$

$p(y) = ly, \sigma(ly) = 3$

$p \vdash \langle \text{while } y < 4, \sigma \rangle \xrightarrow{*} \langle \text{true}, \sigma \rangle$

$p \vdash \langle x, \sigma \rangle \xrightarrow{*} \langle 5, \sigma \rangle$

$p \vdash \langle y := x, \sigma \rangle \xrightarrow{*} \langle y := 5, \sigma \rangle$

$p(x) = 5$

$p \vdash \langle y := 5, \sigma \rangle \xrightarrow{*} \sigma[ly \mapsto 5]$

analisi condizione

$p \vdash \langle y, \sigma \rangle \xrightarrow{*} \langle 5, \sigma \rangle$

$p \vdash \langle y < 4, \sigma \rangle \xrightarrow{*} \langle 5 < 4, \sigma \rangle$

$p(y) = ly, \sigma(ly) = 3$

$p \vdash \langle \text{while } y < 4, \sigma \rangle \xrightarrow{*} \langle \text{false}, \sigma \rangle$

$p = [x \mapsto 5, y \mapsto ly]$

$\sigma = [ly \mapsto 5]$